

AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

```
#include <iostream>

#include <string>

using namespace std;

int main() {

    int num;

    cout << "Enter number of productions: ";

    cin >> num;

    string productions[20];

    for (int i = 0; i < num; i++) {

        cout << "Enter production " << i + 1 << ": ";

        cin >> productions[i];

    }

    cout << "\n--- Eliminating Left Recursion ---\n";

    cout << "Note: '#' represents epsilon (empty string)\n";

    for (int i = 0; i < num; i++) {

        string prod = productions[i];

        char non_terminal = prod[0];

        string right = prod.substr(3);

        string alphas[10], betas[10];

        int alpha_count = 0, beta_count = 0;

        string temp = "";

        for (int k = 0; k < right.length(); k++) {

            if (right[k] == '|') {

                if (temp[0] == non_terminal)

                    alphas[alpha_count++] = temp.substr(1);
```

AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

```
        else

            betas[beta_count++] = temp;

            temp = "";

        } else {

            temp += right[k];

        }

    }

    if (!temp.empty()) {

        if (temp[0] == non_terminal)

            alphas[alpha_count++] = temp.substr(1);

        else

            betas[beta_count++] = temp;

    }

    if (alpha_count > 0) {

        cout << "\nLeft recursion found in production: " << prod << endl;

        cout << "New Grammar:\n";

        if (beta_count == 0) {

            betas[beta_count++] = "#"; // epsilon represented as #

        }

        for (int j = 0; j < beta_count; j++) {

            if (betas[j] == "#")

                cout << non_terminal << " -> " << non_terminal << "\n";

            else

                cout << non_terminal << " -> " << betas[j] <<

non_terminal << "\n";

        }

    }

}
```

AGNEL INSTITUTE OF TECHNOLOGY AND DESIGN

```
}

cout << non_terminal << "" -> ";

for (int j = 0; j < alpha_count; j++) {

    cout << alphas[j] << non_terminal << "";

    if (j != alpha_count - 1)

        cout << " | ";

}

cout << " | #\n";

} else {

    cout << "\nNo left recursion in production: " << prod << endl;

}

}

return 0;

}
```

OUTPUT

<pre>PS C:\Users\knagv> cd "c:\Users\knagv\Desktop\" ; if (\$?) { Enter number of productions: 3 Enter production 1: E->E+T T Enter production 2: T->T*F F Enter production 3: F->(E) id --- Eliminating Left Recursion --- Note: '#' represents epsilon (empty string) Left recursion found in production: E->E+T T New Grammar: E -> TE' E' -> +TE' # Left recursion found in production: T->T*F F New Grammar: T -> FT' T' -> *FT' # No left recursion in production: F->(E) id</pre>	<pre>PS C:\Users\knagv\Desktop> cd "c:\Users\knagv\Desktop\" Enter number of productions: 2 Enter production 1: S->Aa b Enter production 2: A->Ac Sb ε --- Eliminating Left Recursion --- Note: '#' represents epsilon (empty string) No left recursion in production: S->Aa b Left recursion found in production: A->Ac Sb ε New Grammar: A -> SbA' A -> εA' A' -> cA' #</pre>
---	--